

RFQ state machine

I implement one whole-ticket contract: one requester, one selected maker and one escrow for 1-8 binary selections. Each quote covers the complete stake. For stake S and payout $P > S$, the maker reserves $P - S$; acceptance adds the requester's S . The sequence below follows that money into a funded position.

Read top to bottom. Solid arrows: calls / writes. Dashed arrows: returns. Frames: optional or alternative paths.

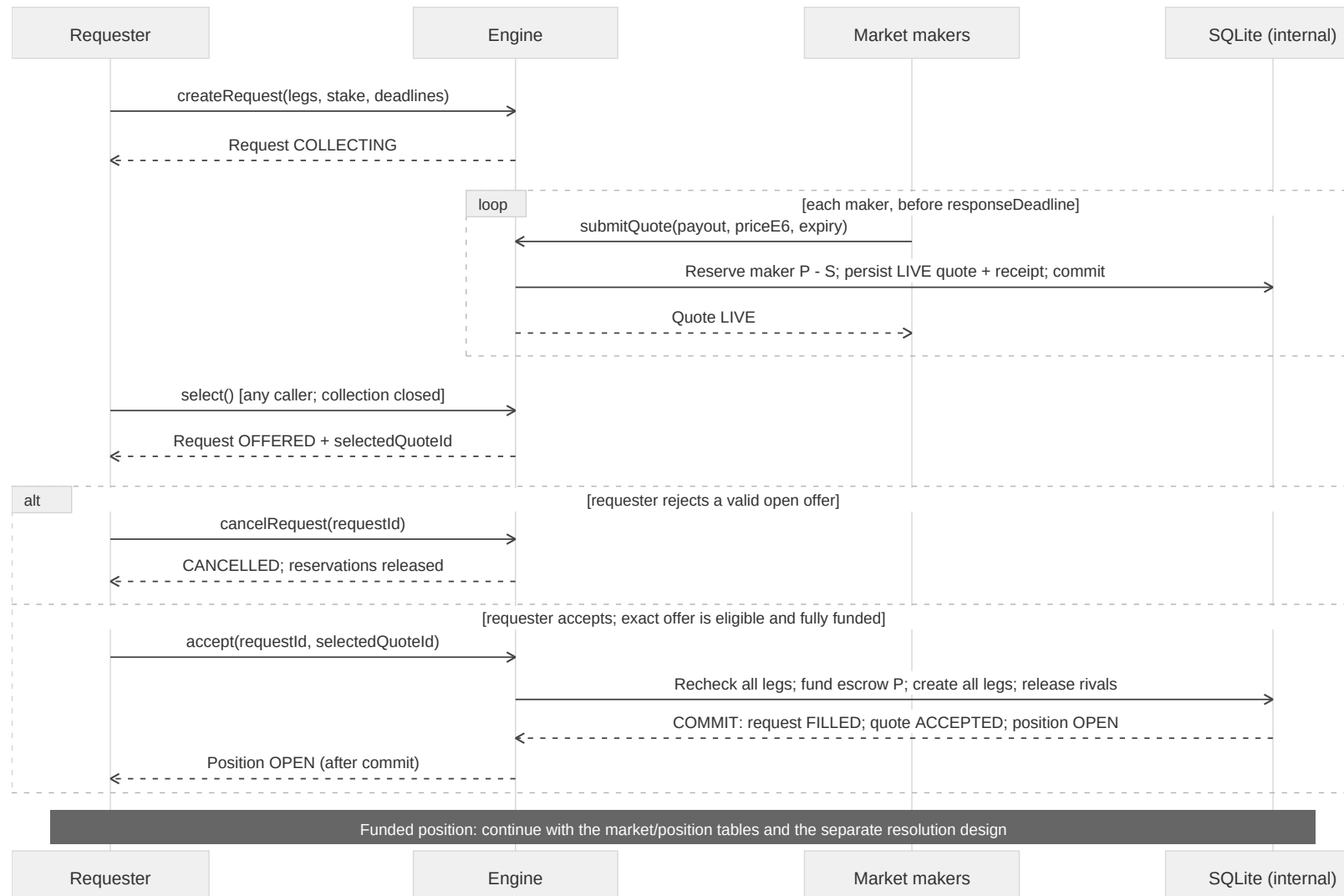


Figure 1. Request to funding. Continue with the complete transition tables; resolution has its own illustrated write-up.

Lifecycle transitions and guarantees

The diagram shows collection, selection and the requester's decision. The tables below add every expiry, rejection and recovery path, then follow the funded position through settlement. The companion resolution design explains that later phase and contains its sequence diagram.

A one-leg request is a single bet. I do not combine makers or multiply leg prices. Amounts are bigint micro-units. The independent-leg interpretation required by the brief is designed at the end and is not implemented.

1. Request: creation through closure

Transition	Caller, condition and money effect
Absent -> COLLECTING	Requester calls <code>createRequest</code> with an unused nonce, immutable selections and <code>now < responseDeadline < acceptanceDeadline <= tradingClosesAt</code> for every market. No requester funds move.
COLLECTING -> OFFERED	Anyone calls <code>select</code> at/after the response deadline. Discard expired quotes; select highest exact payout, then earliest receipt. All surviving maker reservations remain locked.
COLLECTING -> REJECTED	Selection finds no live quote, or refresh finds an invalid leg. Return all remaining reservations.
OFFERED -> FILLED	Requester calls <code>accept</code> for the exact selected quote before expiry. Validate all legs and backing, fund one position, and release every competitor atomically.
OFFERED -> REJECTED	Refresh finds an invalid leg, or acceptance finds insufficient requester funds. No position; return all reservations.
Either open state -> EXPIRED	Refresh sees the acceptance deadline reached, or an offered quote expired. Return all reservations.
Either open state -> CANCELLED	Requester calls <code>cancelRequest</code> , including to reject an offer. Applicable expiry/invalidity takes precedence; return all reservations.
FILLED -> SETTLED	Anyone calls <code>settle</code> , or recovery settles, once every market is final. Pay or refund the position in the same transaction.

Refresh and deadlines. `select` and `recover` are permissionless mock calls; requester acceptance/cancellation and maker quote submission/cancellation also refresh after their initial checks. Refresh checks the acceptance deadline first, selected-quote expiry second, then invalid markets: halted, no longer unresolved, trading cutoff reached, or changed terms hash. Reads, market halt and result proposals do not sweep RFQs. Time alone changes no stored state.

At the response deadline, selection becomes eligible and quote submission/ordinary withdrawal close. Acceptance must be strictly before the request deadline, selected quote expiry and every market's trading cutoff.

REJECTED, EXPIRED, CANCELLED and SETTLED are terminal. Request IDs and requester nonces remain consumed. An offer is never silently reselected; a new offer requires a new request. A funded request cannot be cancelled.

Lifecycle transitions and guarantees / continued

2. Quotes: reservation through release

Transition	Caller, condition and money effect
Absent -> LIVE	Maker calls <code>submitQuote</code> before collection closes, committing to the request hash and full stake. Require consistent price/payout and <code>now < expiresAt <= acceptanceDeadline</code> ; reserve <code>P - S</code> . Self-quotes are rejected.
LIVE -> SELECTED	<code>select</code> chooses the winner; its reservation remains locked. Rival quotes stay LIVE and reserved.
Ordinary LIVE -> CANCELLED	Its maker calls <code>cancelQuote</code> before the response deadline. Return its reservation.
LIVE -> EXPIRED	Successful submission/selection cleanup, its maker's cancellation call, or recovery observes its expiry; whole-request expiry also expires every remaining quote. Return its reservation.
SELECTED -> EXPIRED	Selected-quote or request expiry closes the whole offer. Every remaining quote becomes EXPIRED, including later-expiring rivals.
LIVE OR SELECTED -> REJECTED	Request rejection/cancellation returns all reservations. Successful acceptance rejects and refunds competing live quotes.
Competing LIVE -> REJECTED	A qualifying replacement at capacity refunds it atomically with reserving the new quote.
SELECTED -> ACCEPTED	Requester acceptance transfers its reservation into the position's escrow. This quote status never changes again.

One maker may have one active quote per request; limits are 32 active quotes and 32 open requests per requester. Ordinary quotes may be cancelled and resubmitted during collection. At the response deadline, all survivors become firm even if nobody has called `select`.

At capacity, an incoming quote must improve payout strictly and preserve the displaced quote's expiry coverage. Choose the lowest eligible payout, breaking ties against the latest receipt. The replacement reserves its full liability and is firm immediately; its immutable `replacesQuoteId` records the refunded predecessor. Underfunding or an exception rolls everything back. Equal payouts or shorter expiry cannot displace a quote; this is not general Sybil protection.

3. Markets: proposal through finality

Trusted bootstrap loads balances and immutable markets once; markets begin UNRESOLVED. Oracle, arbiter, wording, trading cutoff, resolution time, dispute period, positive adjudication period and fallback are committed before quoting. Caller strings represent already-authenticated mock identities.

Transition	Caller and exact guard; no collateral moves
UNRESOLVED -> PROPOSED	Designated oracle calls <code>proposeResult</code> or validated <code>proposeHip4Result</code> at/after <code>resolveAfter</code> ; require <code>now + disputePeriod + adjudicationPeriod <= fallbackAt</code> . Record result/evidence and <code>challengeDeadline = now + disputePeriod</code> .
PROPOSED -> DISPUTED	Requester or maker of an open position containing the market calls <code>disputeResult</code> strictly before <code>challengeDeadline</code> . Record the reason; deadlines do not move.
PROPOSED -> FINAL	Anyone calls <code>finalizeResult</code> , or recovery acts, at/after the challenge deadline. Preserve the proposed result even if the caller arrives after fallback.

Lifecycle transitions and guarantees / continued

Transition	Caller and exact guard; no collateral moves
DISPUTED -> FINAL	Designated arbiter calls <code>arbitrateResult</code> strictly before fallback, recording YES, NO or VOID. It may act immediately; the adjudication period reserves time, not a mandatory waiting period.
UNRESOLVED OR DISPUTED -> FINAL	Anyone calls <code>finalizeResult</code> , or recovery acts, at/after fallback. Record VOID. Final results cannot be revised.

The operator alone may call `haltMarket`. Halt is a separate irreversible flag, not a resolution state. It blocks new requests and makes open RFQs ineligible on their next refresh; it does not refund them immediately or change funded positions' resolution path.

4. Position: escrow through payout

Transition	Trigger and money effect
Absent -> OPEN	Successful acceptance creates all position legs and funds escrow of P. Request becomes FILLED; selected quote becomes ACCEPTED.
OPEN -> LOST	Anyone settles after all legs are FINAL; any result opposite the selected side pays P to the maker, even if another leg is VOID.
OPEN -> VOID	All legs are final, none lost and at least one is VOID: refund requester S and maker P - S.
OPEN -> WON	All legs are final and every selected side won: pay P to the requester.

Any caller may trigger these settlement transitions with `settle` or `recover`. Each settlement empties escrow and moves the request to SETTLED atomically. No individual leg pays independently. A known loss still waits for the other legs' final results. Settling an already terminal position returns it without transferring again.

5. Architecture, partial failure and recovery

Engine delegates RFQs to `src/rfq/` and results/payments to `src/resolution/`. `CommandContext` shares one Store; `CommandExecutor` acquires SQLite's BEGIN IMMEDIATE lock, checks the receipt, then reads time for a new command. New command time cannot precede the last committed time. State, balances, ledger, events and receipt share one synchronous commit. These are modules around one execution authority, not separately committing services.

Implemented three-leg acceptance. `src/rfq/acceptance.ts` validates every leg, then creates the position, transfers requester stake and maker reservation, copies all legs, consumes the request and refunds competitors. If leg 2 is invalid, commit whole-request rejection and reservation releases. If execution fails after writing leg 1 but before commit, roll back all attempted writes; the original offer and backing remain retryable. No partial position survives.

Errors and repeats. Invalid input/authority, wrong selected quote, insufficient maker backing and pre-commit exceptions roll back the command, including attempted cleanup. Business rejection/expiry deliberately commits releases. Identical (actor, commandId, payload) retries return the historical receipt before clock or lifecycle checks; conflicting input fails. A lost post-commit response is recovered by replay. Fresh acceptance/cancellation of a FILLED or SETTLED request fails. After refresh, `select` returns a non-collecting request unchanged; cancellation returns already closed requests/quotes unchanged. `finalizeResult` and `settle` return already final markets/positions without another payment; arbitration cannot change a final outcome.

Lifecycle transitions and guarantees / continued

Liveness. Any caller may invoke bounded `recover` (1-1,000 work items). A durable cursor rotates request cleanup, quote expiry, market finalization and settlement. Retry a failed response with the same command ID; use a new ID for the next batch. Recovery never selects offers. With an available database, advancing trusted time and a caller, fixed fallback provides an eventual payout/refund route.

Independently matched legs (design only)

For the brief's per-leg interpretation, I would commit separate notionals/payoffs and reserve each maker's full per-leg liability. After collection, anyone could select the best live, full-size quote for each leg inside one transaction. Provisional choices would not be separately committed. Each quote has its own expiry; only its maker could cancel during collection.

Leg 1 provisionally matches; leg 2 of 3 cannot fill: reject the entire basket and refund every reservation across all three legs. No requester debit or position exists. Committing leg 1 early, forgetting rival reservations or abandoning provisional work would leak capital; atomic rejection and deadline recovery prevent those paths.

Only a complete vector becomes `OFFERED`. An open basket would become `CANCELLED` at the requester's instruction, `EXPIRED` at the request or selected-leg expiry, or `REJECTED` on invalidity, releasing every reservation without reselection. Requester acceptance before every expiry would recheck all legs/backing, fund every position, release every rival, consume the basket and save its receipt in one commit. Insufficient funds/invalidity rejects all; pre-commit failure rolls back all. Each funded leg would settle independently; the basket becomes `SETTLED` when all are terminal. This requires a separate API/schema and tests; the running engine implements whole-ticket matching.